

Abstract Interpretation (1)

CSE552 Program Analysis — Lecture 18

Jaemin Hong

Formalizing Soundness

- ▶ So far, we have used the term *soundness* informally
 - ▶ If an analysis is sound, the properties it infers for a given program hold in all actual executions of the program
- ▶ Abstract interpretation¹
 - ▶ Provides a mathematical foundation for what it means for an analysis to be sound
 - ▶ Relates the analysis specification to the formal semantics of the programming language

¹Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints (Cousot and Cousot, 1977)

Concrete vs. Abstract Semantics

- ▶ Concrete semantics: captures the meaning of programs in ordinary execution without any approximations
 - ▶ The usual notion of semantics
 - ▶ Denotational semantics, operational semantics, axiomatic semantics, etc.
- ▶ Abstract semantics: the approximated meaning of programs used by the static analysis under consideration
 - ▶ What we have defined using constraint rules

Collecting Semantics

- Concrete state:

$$CState = Var \rightarrow \mathbb{Z}$$

- For each CFG node v , $\{v\}$ is a constraint variable that ranges over sets of concrete states:

$$\{v\} \in \mathcal{P}(CState)$$

- $\{v\}$ shall denote the set of concrete states that are possible at the program point immediately after the instruction represented by v , in some execution of the program

Expression Evaluation

$$ceval : CState \times Expr \rightarrow \mathcal{P}(\mathbb{Z})$$

$$ceval(\rho, x) = \{\rho(x)\}$$

$$ceval(\rho, n) = \{n\}$$

$$ceval(\rho, \text{input}()) = \mathbb{Z}$$

$$ceval(\rho, e_1 + e_2) = \{z_1 + z_2 \mid z_1 \in ceval(\rho, e_1) \wedge z_2 \in ceval(\rho, e_2)\}$$

$$ceval : \mathcal{P}(CState) \times Expr \rightarrow \mathcal{P}(\mathbb{Z})$$

$$ceval(R, e) = \bigcup_{\rho \in R} ceval(\rho, e)$$

Successor Nodes

$$csucc : CState \times Node \rightarrow \mathcal{P}(Node)$$

v : if e

$$csucc(\rho, v) = T \cup F$$

where

$$T = \begin{cases} \{true(v)\} & \text{if } z \in ceval(\rho, e) \text{ for some } z \neq 0 \\ \emptyset & \text{otherwise} \end{cases}$$

$$F = \begin{cases} \{false(v)\} & \text{if } 0 \in ceval(\rho, e) \\ \emptyset & \text{otherwise} \end{cases}$$

v : other nodes

$$csucc(\rho, v) = succ(v)$$

$$csucc : \mathcal{P}(CState) \times Node \rightarrow \mathcal{P}(Node)$$

$$csucc(R, v) = \bigcup_{\rho \in R} csucc(\rho, v)$$

Join

$$CJOIN(v) = \{\rho \in CState \mid \exists w \in Node. \rho \in \llbracket w \rrbracket \wedge v \in csucc(\rho, w)\}$$

Constraint Rules

$v : x = e$

$$\{v\} = \{\rho[x \mapsto z] \mid \rho \in CJOIN(v) \wedge z \in ceval(\rho, e)\}$$

v : entry

$$\{v\} = \{[x_0 \mapsto z_0, \dots] \mid z_i \in \mathbb{Z}\}$$

v : other nodes

$$\{v\} = CJOIN(v)$$

Lattices

- ▶ $(\mathcal{P}(\mathbb{Z}), \subseteq)$ is a lattice
- ▶ $(\mathcal{P}(CState), \subseteq)$ is a lattice
- ▶ $\mathcal{P}(CState)^n$ is a lattice
 - ▶ $n = |Node|$

Semantics of Program

$$\{v_1\} = cf_1(\{v_1\}, \{v_2\}, \dots, \{v_n\})$$

$$\{v_2\} = cf_2(\{v_1\}, \{v_2\}, \dots, \{v_n\})$$

$$\vdots$$

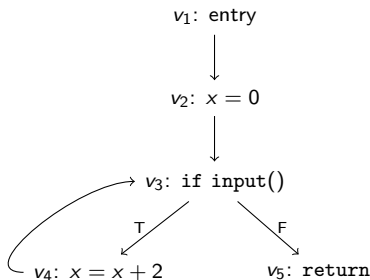
$$\{v_n\} = cf_n(\{v_1\}, \{v_2\}, \dots, \{v_n\})$$

$$cf : \mathcal{P}(CState)^n \rightarrow \mathcal{P}(CState)^n$$

$$cf((x_1, \dots, x_n)) = (cf_1(x_1, \dots, x_n), \dots, cf_n(x_1, \dots, x_n))$$

- ▶ The least fixed point for cf describes the semantics of the program
 - ▶ The set of all concrete states that can occur at each program point in some execution of the program

Example: Concrete Semantics of a Loop



Constraints:

$$\{v_1\} = \{[x \mapsto z] \mid z \in \mathbb{Z}\}$$

$$\{v_2\} = \{\rho[x \mapsto 0] \mid \rho \in \{v_1\}\}$$

$$\{v_3\} = \{v_2\} \cup \{v_4\}$$

$$\{v_4\} = \{\rho[x \mapsto \rho(x) + 2] \mid \rho \in \{v_3\}\}$$

$$\{v_5\} = \{v_3\}$$

Least fixed point:

$$\{v_1\} = \{[x \mapsto z] \mid z \in \mathbb{Z}\}$$

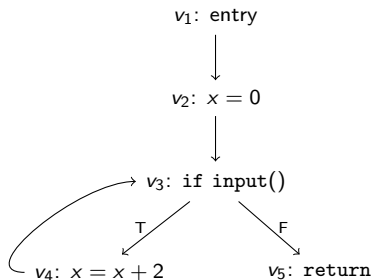
$$\{v_2\} = [x \mapsto 0]$$

$$\{v_3\} = \{[x \mapsto z] \mid z \in \{0, 2, 4, \dots\}\}$$

$$\{v_4\} = \{[x \mapsto z] \mid z \in \{2, 4, 6, \dots\}\}$$

$$\{v_5\} = \{[x \mapsto z] \mid z \in \{0, 2, 4, \dots\}\}$$

Why Least Fixed Point?



Constraints:

$$\{v_1\} = \{[x \mapsto z] \mid z \in \mathbb{Z}\}$$

$$\{v_2\} = \{\rho[x \mapsto 0] \mid \rho \in \{v_1\}\}$$

$$\{v_3\} = \{v_2\} \cup \{v_4\}$$

$$\{v_4\} = \{\rho[x \mapsto \rho(x) + 2] \mid \rho \in \{v_3\}\}$$

$$\{v_5\} = \{v_3\}$$

Another fixed point:

$$\{v_1\} = \{[x \mapsto z] \mid z \in \mathbb{Z}\}$$

$$\{v_2\} = [x \mapsto 0]$$

$$\{v_3\} = \{[x \mapsto z] \mid z \in \mathbb{Z}\}$$

$$\{v_4\} = \{[x \mapsto z] \mid z \in \mathbb{Z}\}$$

$$\{v_5\} = \{[x \mapsto z] \mid z \in \mathbb{Z}\}$$

Existence of the Least Fixed Point

- ▶ Since cf is monotone, the lfp exists according to Tarski's fixed-point theorem
- ▶ $\mathcal{P}(CState)^n$ has an infinite height
- ▶ In general, the lfp is non-computable

Notations

- ▶ $\{P\} = \text{lfp}(cf)$ where cf is the semantic constraint function
 - ▶ $\{P\}$ denotes the (concrete) semantics of P
- ▶ $\llbracket P \rrbracket = \text{lfp}(f)$ where f is the analysis constraint function
 - ▶ $\llbracket P \rrbracket$ denotes the analysis result (abstract semantics) of P

Abstraction Functions

- Define how each element from the semantic lattice is most precisely described by an element in the analysis lattice

$$\alpha_a : \mathcal{P}(\mathbb{Z}) \rightarrow \text{Sign}$$
$$\alpha_a(D) = \begin{cases} \perp & \text{if } D = \emptyset \\ + & \text{if } D \neq \emptyset \wedge \forall z \in D. z > 0 \\ - & \text{if } D \neq \emptyset \wedge \forall z \in D. z < 0 \\ 0 & \text{if } D = \{0\} \\ \top & \text{otherwise} \end{cases}$$

Abstraction Functions (cont.)

$$\alpha_b : \mathcal{P}(CState) \rightarrow State$$

$$\alpha_b(R) = \sigma \text{ where } \sigma(x) = \alpha_a(\{\rho(x) \mid \rho \in R\})$$

(where $State = Var \rightarrow Sign$)

$$\alpha_c : \mathcal{P}(CState)^n \rightarrow State^n$$

$$\alpha_c((R_1, \dots, R_n)) = (\alpha_b(R_1), \dots, \alpha_b(R_n))$$

Monotonicity of Abstraction

- ▶ Abstraction functions should be monotone
 - ▶ A larger set of concrete elements should not be represented by a smaller abstract element in the lattice order

Concretization Functions

- Express the meaning of the analysis lattice elements in terms of the semantic lattice elements

$$\gamma_a : \textit{Sign} \rightarrow \mathcal{P}(\mathbb{Z})$$

$$\gamma_a(\perp) = \emptyset$$

$$\gamma_a(+)= \{z \in \mathbb{Z} \mid z > 0\}$$

$$\gamma_a(-)= \{z \in \mathbb{Z} \mid z < 0\}$$

$$\gamma_a(0) = \{0\}$$

$$\gamma_a(\top) = \mathbb{Z}$$

Concretization Functions (cont.)

$$\gamma_b : \textit{State} \rightarrow \mathcal{P}(\textit{CState})$$

$$\gamma_b(\sigma) = \{\rho \mid \forall x. \rho(x) \in \gamma_a(\sigma(x))\}$$

$$\gamma_c : \textit{State}^n \rightarrow \mathcal{P}(\textit{CState})^n$$

$$\gamma_c((\sigma_1, \dots, \sigma_n)) = (\gamma_b(\sigma_1), \dots, \gamma_b(\sigma_n))$$

Monotonicity of Concretization

- ▶ Concretization functions should be monotone
 - ▶ A greater abstract value should not express a smaller set of concrete elements

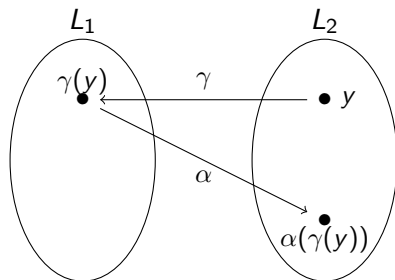
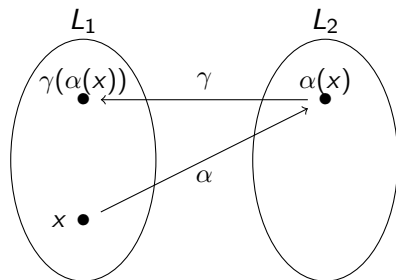
Galois Connections

- ▶ L_1 and L_2 are complete lattices
- ▶ $\alpha : L_1 \rightarrow L_2$ is an abstraction function
- ▶ $\gamma : L_2 \rightarrow L_1$ is a concretization function
- ▶ α and γ should satisfy the following properties:
 - ▶ $\gamma \circ \alpha$ is extensive (i.e., $\forall x \in L_1. x \sqsubseteq \gamma(\alpha(x))$)
 - ▶ $\alpha \circ \gamma$ is reductive (i.e., $\forall y \in L_2. \alpha(\gamma(y)) \sqsubseteq y$)
- ▶ The pair of monotone functions (α, γ) is called a Galois connection

Intuitions

- ▶ $x \sqsubseteq \gamma(\alpha(x))$
 - ▶ Abstraction may lose precision but must be safe
- ▶ $\alpha(\gamma(y)) \sqsubseteq y$
 - ▶ Abstraction should always give the most precise possible description

Intuitions, Visually



Example: Sign Galois Connection (Extensive)

- ▶ $\gamma(\alpha(\emptyset)) = \gamma(\perp) = \emptyset \supseteq \emptyset$
- ▶ $\gamma(\alpha(\{1\})) = \gamma(+)= \{z \in \mathbb{Z} \mid z > 0\} \supseteq \{1\}$
- ▶ $\gamma(\alpha(\{0\})) = \gamma(0) = \{0\} \supseteq \{0\}$
- ▶ $\gamma(\alpha(\{-1\})) = \gamma(-) = \{z \in \mathbb{Z} \mid z < 0\} \supseteq \{-1\}$
- ▶ $\gamma(\alpha(\{1, -1\})) = \gamma(\top) = \mathbb{Z} \supseteq \{1, -1\}$
- ▶ $\gamma \circ \alpha$ is typically not the identity function
 - ▶ Abstraction may lose precision

Example: Sign Galois Connection (Reductive)

- ▶ $\alpha(\gamma(\perp)) = \alpha(\emptyset) = \perp \sqsubseteq \perp$
- ▶ $\alpha(\gamma(+)) = \alpha(\{z \in \mathbb{Z} \mid z > 0\}) = + \sqsubseteq +$
- ▶ $\alpha(\gamma(-)) = \alpha(\{z \in \mathbb{Z} \mid z < 0\}) = - \sqsubseteq -$
- ▶ $\alpha(\gamma(0)) = \alpha(\{0\}) = 0 \sqsubseteq 0$
- ▶ $\alpha(\gamma(\top)) = \alpha(\mathbb{Z}) = \top \sqsubseteq \top$

- ▶ $\alpha \circ \gamma$ is typically the identity function

When is $\alpha \circ \gamma$ Not the Identity?

- ▶ When is $\alpha \circ \gamma$ not the identity function?
- ▶ $\gamma(\alpha(\gamma(x))) = \gamma(x)$ for all x
 - ▶ $\gamma(x) \sqsubseteq \gamma(\alpha(\gamma(x)))$
 - ▶ $\alpha(\gamma(x)) \sqsubseteq x$ and γ is monotone, so $\gamma(\alpha(\gamma(x))) \sqsubseteq \gamma(x)$
- ▶ Even if $\alpha(\gamma(x)) \neq x$, they have the same concretization
- ▶ If two abstract elements have the same concretization, $\alpha \circ \gamma$ may not be the identity function

Unique Determination

If L_1 and L_2 are complete lattices and $\alpha : L_1 \rightarrow L_2$ and $\gamma : L_2 \rightarrow L_1$ form a Galois connection, then

- ▶ γ is uniquely determined by α :

$$\gamma(y) = \bigsqcup \{x \in L_1 \mid \alpha(x) \sqsubseteq y\}$$

- ▶ α is uniquely determined by γ :

$$\alpha(x) = \bigsqcap \{y \in L_2 \mid x \sqsubseteq \gamma(y)\}$$

Implication

- ▶ Once the analysis designer has specified the concrete and abstract semantics, the relation between the concrete domain and the abstract domain may be specified
 - ▶ using an abstraction function α
 - ▶ or using a concretization function γ
- ▶ This holds only when α and γ form a Galois connection
- ▶ Question: When does α have γ such that they form a Galois connection, and vice versa?

Complete Join/Meet Morphisms

- ▶ Where L_1 and L_2 are complete lattices, $f : L_1 \rightarrow L_2$ is a complete join morphism if $f(\bigsqcup A) = \bigsqcup_{a \in A} f(a)$ for all $A \subseteq L_1$
- ▶ Where L_1 and L_2 are complete lattices, $f : L_1 \rightarrow L_2$ is a complete meet morphism if $f(\bigsqcap A) = \bigsqcap_{a \in A} f(a)$ for all $A \subseteq L_1$

Existence of Galois Connections

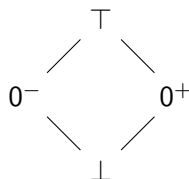
- ▶ If α is a complete join morphism, then there exists γ such that α and γ form a Galois connection
- ▶ If γ is a complete meet morphism, then there exists α such that α and γ form a Galois connection

Galois Connection as Sanity Check

- ▶ A certain analysis lattice may not have a Galois connection with the semantic lattice
- ▶ This does not necessarily mean that the design is wrong
 - ▶ The analysis still may be sound
 - ▶ The analysis still can terminate
- ▶ However, the analysis may produce surprising results

Example: A Lattice Without a Galois Connection

$$\text{Sign}' = \{\perp, 0^-, 0^+, \top\}$$



$$\gamma(\perp) = \emptyset$$

$$\gamma(0^-) = \{z \in \mathbb{Z} \mid z \leq 0\}$$

$$\gamma(0^+) = \{z \in \mathbb{Z} \mid z \geq 0\}$$

$$\gamma(\top) = \mathbb{Z}$$

- ▶ How should we define $\text{eval}(\sigma, 0)$?
- ▶ How should we define $\alpha(\{0\})$?

Example: No Galois Connection Exists

- ▶ Let $\alpha(\{0\}) = 0^-$
 - ▶ $\{0\} \sqsubseteq \{0, 1\}$, but $0^- \not\sqsubseteq 0^+$
- ▶ Let $\alpha(\{0\}) = 0^+$
 - ▶ $\{0\} \sqsubseteq \{-1, 0\}$, but $0^+ \not\sqsubseteq 0^-$
- ▶ No Galois connection exists between $Sign'$ and $\mathcal{P}(\mathbb{Z})$ with the given γ

Example: Counter-Intuitive Imprecision

```
x = e1 != e2; // e1 always equals e2
// is x non-negative?
```

- ▶ Initially, the analysis fails to recognize that e_1 always equals e_2 , so it assigns 0^+ to x , proving that x is non-negative
- ▶ After improving the analysis, it recognizes that e_1 always equals e_2 , so it assigns 0^- to x , failing to prove that x is non-negative
- ▶ More precise analysis can lead to less precise results, which is counter-intuitive

Representation Functions

- ▶ We can also specify the connection between concrete elements and abstract elements using a representation function
- ▶ Example: Sign

$$\beta : \mathbb{Z} \rightarrow \textit{Sign}$$

$$\beta(z) = \begin{cases} 0 & \text{if } z = 0 \\ + & \text{if } z > 0 \\ - & \text{if } z < 0 \end{cases}$$

- ▶ α can be derived from β :

$$\alpha(D) = \bigsqcup_{z \in D} \beta(z)$$

Deriving Galois Connections from Representation Functions

- ▶ Let $\beta : V \rightarrow L$ where V is a set and L is a complete lattice
- ▶ $\alpha : \mathcal{P}(V) \rightarrow L$

$$\alpha(S) = \bigsqcup_{v \in S} \beta(v)$$

- ▶ $\gamma : L \rightarrow \mathcal{P}(V)$
$$\gamma(x) = \{v \in V \mid \beta(v) \sqsubseteq x\}$$

- ▶ Then, α and γ form a Galois connection

Galois Connections and Map Lattices

- ▶ Let L_1 and L_2 be complete lattices, S be a set, and $\alpha : L_1 \rightarrow L_2$ and $\gamma : L_2 \rightarrow L_1$ form a Galois connection
- ▶ Then, following α' and γ' form a Galois connection between $S \rightarrow L_1$ and $S \rightarrow L_2$:

- ▶ $\alpha' : (S \rightarrow L_1) \rightarrow (S \rightarrow L_2)$

$$\alpha'(x)(s) = \alpha(x(s))$$

- ▶ $\gamma' : (S \rightarrow L_2) \rightarrow (S \rightarrow L_1)$

$$\gamma'(y)(s) = \gamma(y(s))$$

Galois Connections and Product Lattices

- ▶ Let L_1 , L_2 , L'_1 , and L'_2 be complete lattices, and $\alpha : L_1 \rightarrow L_2$ and $\gamma : L_2 \rightarrow L_1$ form a Galois connection, and $\alpha' : L'_1 \rightarrow L'_2$ and $\gamma' : L'_2 \rightarrow L'_1$ form a Galois connection
- ▶ Then, following α'' and γ'' form a Galois connection between $L_1 \times L'_1$ and $L_2 \times L'_2$:
 - ▶ $\alpha'' : (L_1 \times L'_1) \rightarrow (L_2 \times L'_2)$

$$\alpha''((x, x')) = (\alpha(x), \alpha'(x'))$$

- ▶ $\gamma'' : (L_2 \times L'_2) \rightarrow (L_1 \times L'_1)$

$$\gamma''((y, y')) = (\gamma(y), \gamma'(y'))$$

Summary

- ▶ Abstract interpretation provides a mathematical foundation for the soundness of static analyses by relating abstract semantics to concrete semantics
- ▶ The concrete semantics of a program can be defined as the least fixed point of a system of constraints over $\mathcal{P}(CState)^n$
- ▶ Abstraction (α) and concretization (γ) functions connect the semantic and analysis lattices
- ▶ α and γ form a Galois connection when they are monotone, $\gamma \circ \alpha$ is extensive, and $\alpha \circ \gamma$ is reductive